Ethnus - Microsoft Azure Fundamentals (AZ-900)

A REPORT

submitted by

Person_001 [USERNAME])

in partial fulfilment for the award

of

B. Tech. Computer Science and Engineering

School of Computer Science and Engineering



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October - 2026



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

DECLARATION

I hereby declare that the project entitled **"Ethnus - Microsoft Azure Fundamentals (AZ-900) "** submitted by me to the School of Computer Science and Engineering, [LOCATION] Institute of Technology, [LOCATION] Campus, [ADDRESS] [LOCATION] in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology – Computer Science and Engineering** is a record of bonafide work carried out by me**.** I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or university.

Signature

Person_001 [USERNAME])



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

CERTIFICATE

The project report entitled "**Ethnus - Microsoft Azure Fundamentals (AZ-900)**" is prepared and submitted by Person_001 **(Register No:** [ID_NUMBER] **)**. It has been found satisfactory in terms of scope, quality and presentation as partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology – Computer Science and Engineering** in [ADDRESS] Institute of Technology, [LOCATION], India.

Examined by:

Dr. Person_002 **P. N.**



eTHNUS

Explore | Expand | Enrich

CERTIFICATE

OF COMPLETION

This is to certify that

Karan Veer Singh

bearing Reg. No. : 24BYB1164 has successfully completed

Microsoft Certified: Azure Fundamentals (AZ-900) Course

May 2026 to **July 2026**

July 15, 2026

Date of Issue

Goutham Nanjundaswamy

Chief Technology Officer

Certificate ID: 62R5SFY5 [Validate at https://ethnus.com/certverify]

Explore | Expand | Enrich

Internship/Certification Program Scorecard

Name	Karan Veer Singh
Registration No	24BYB1164
Email	karan.veersingh2024@vitstudent.ac.in
Course	Azure Fundamentals (AZ-900) Course
Batch	04:00 pm to 06:00 pm
Trainer	Varun
Evaluation Date	15-Jul-26
Certificate ID	62R5SFY5

Evaluation

Criteria	Maximum Marks	Marks Obtained
Assignments - 01	25	20
Assignments - 02	25	23
Final Project / Assessment	50	48
Total	**100**	**91**
Grade	A	

Grade Reference

A	B	C	D
Excellent (85–100%)	Good (70–84%)	Average (50–69%)	Needs Improvement (<50%)

62r5sfy5:1a12f2ab0a9fc5cdb97cc2ebeaf4dd52

Explore | Expand | Enrich

Issued by Ethnus on 27 Jul 2026. Document ID: 62r5sfy5:1a12f2ab0a9fc5cdb97cc2ebeaf4dd52
© 2026 Ethnus. All rights reserved. | www.ethnus.com

Microsoft AZ-900

Microsoft Azure Fundamentals



Candidate: Karan Veer Singh
Date: Saturday, July 18, 2026
Candidate ID: MS1101386807
Site Number: Proctored
Result: PASS
Registration: 541305566
Passing Score: 700
Your Score: 921



ACKNOW EMENT

I would like to express my sincere gratitude to Person_003 for providing the opportunity to undertake learning and certification preparation in Micro re Fundamentals (AZ-900) during 18 May 2026 to 10 July 2026. The programme helped me develop a structured understanding of cloud concepts, Azure architecture and services, and Azure management and governance.

I also acknowledge the g and academic support by the School of Computer Science and Engineering, [LOCATION] Institute of Technology, [LOCATION] . I am thankful to the faculty members and my family f ncouragement during thi period.

- **Dr.** Person_004 of the Department, B. mputer Science and Engineering (Da [LOCATION] Institute of Technology, [LOCATION]
- **Dr.** Person_005 , School of Computer Science & Engineering, [LOCATION] Inst [LOCATION]
- **HYANANDAM P**, Ass an, School of Computer Science & Engineering, [LOCATION] Institute of Technology, [LOCATION]
- **Dr. SUGANYA G**, Ass an, School of Computer Science & Engineering, [LOCATION] Institute of Technology, [LOCATION]
- **Dr.** Person_006 **,** Assoc n, School of Computer Science & Eng [LOCATION] ology, [LOCATION]

CONTENTS

Chapter	Title	Page
	Title Page	i
	Declaration	ii
	Certificate	iii
	Industry certificate	iv - vi
	Acknowledgement	vii
	Table of contents	viii
	List of Abbreviation	ix
	Abstract	x - xi
1	Introduction	01
2	Azure Cloud Architecture & Fundamentals	02
3	System Implementation & Workflow	03
4	Conclusion	04
	References	05
	Appendix – I	06 - 12

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
AZ-900	Microsoft Azure Fundamentals
CBIMS	Cloud-Based Inventory Management System
CLI	Command Line Interface
DB	Database
IaaS	Infrastructure as a Service
IAM	Identity and Access Management
NoSQL	Not Only SQL (Non-relational database)
PaaS	Platform as a Service
RBAC	Role-Based Access Control
SDK	Software Development Kit
SKU	Stock Keeping Unit
URI	Uniform Resource Identifier

ABSTRACT

- **Present scenario of the area**

In modern supply chain and retail operations, effective inventory control is vital for business scalability and logistical accuracy. Most enterprises currently track items, manage warehouses, and record transactions using computer-based inventory systems. However, a significant majority of traditional setups remain deployed on on-premises infrastructure. In this prevailing model, organizations host local database servers, web interfaces, and manual file directories within private physical facilities. These architectures are typically monolithic, tightly coupling inventory database records, file and image storage for product assets, and alert processing routines inside single, centralized host machines managed entirely by internal IT personnel.

- **Limitations and Pitfalls of the Present Scenario**

Traditional on-premises architectures impose severe operational and technical bottlenecks:

- **Poor Scalability and High Cost:** Expanding capacity requires purchasing expensive physical server hardware, resulting in high capital expenditure and underutilized resources during off-peak cycles.
- **Single Points of Failure:** Localized server or network disruptions can cause complete system outages, inventory discrepancies, and permanent data loss without extensive, costly disaster recovery setups.
- **Data Silos and Restricted Access:** Synchronizing inventory across multiple warehouse locations in real time is difficult, restricting cross-facility visibility and delaying stock audits.
- **Synchronous Processing Bottlenecks:** Tightly coupled monolithic systems struggle when handling resource-intensive background jobs, such as generating real-time low-stock alerts and processing batch updates, which degrades core transactional performance.

- **Problem Addressed**

This project addresses the critical need for a resilient, highly available, and scalable inventory management architecture that eliminates physical hardware dependency. Specifically, it tackles rigid relational database schemas that complicate diverse product catalog updates, the inefficiency of storing asset images on local disks, synchronous task bottlenecks during high-volume alert processing, and the security challenges of managing user credentials securely in distributed environments.

- **Proposed Solution**

To solve these challenges, this project introduces a Cloud-Based Inventory Management System built on Microsoft Azure Platform-as-a-Service (PaaS) offerings, developed as part of the Ethnus-Microsoft Azure Fundamentals (AZ-900) internship:

- **Dynamic Data Storage:** Azure Cosmos DB is employed as a multi-region, low-latency NoSQL document database, enabling schema-agnostic catalog management and fast CRUD operations for diverse stock-keeping units (SKUs).
- **Decoupled Media Management:** Azure Blob Storage is integrated to store unstructured assets, including uploaded product photos and barcodes generated dynamically via Python's barcode library.

- **Asynchronous Alert Queuing:** Azure Queue Storage decouples background workflows, ensuring low-stock thresholds trigger asynchronous notifications without throttling the main user application.
- **Cloud & Local Security:** Identity and access workflows utilize Azure Identity for secure cloud service connections, w pt library enforces robust cryptographic hashing for administrative credential [PASSWORD]

1 INTRODUCTION

In the era of digital transformation, supply chain logistics and inventory management have evolved from simple ledger-based tracking to complex, data-driven ecosystems. Historically, organizations relied heavily on traditional on-premises systems to monitor their stock levels, manage warehouse operations, and facilitate order fulfillments. While these legacy systems functioned adequately for localized operations, they are increasingly insufficient for modern, dynamic business environments. On-premises infrastructures typically require substantial upfront capital investments, dedicated hardware maintenance, and complex disaster recovery setups. Furthermore, they often operate as siloed environments, leading to latency in data synchronization, restricted cross-location accessibility, and severe difficulties in scaling during periods of rapid business growth or sudden demand spikes.

The advent of Cloud Computing has revolutionized this landscape by offering Infrastructure as a Service (IaaS) and Platform as a Service (PaaS) models. By migrating inventory systems to the cloud, organizations can leverage on-demand scalability, high availability, and globally distributed data centers. Microsoft Azure, a leading enterprise-grade cloud platform, provides a robust suite of services that allows developers to build resilient, serverless applications without the administrative overhead of managing physical hardware. This project, undertaken during the Ethnus-Microsoft Azure Fundamentals (AZ-900) internship, explores the practical transition from legacy inventory methods to a modernized, cloud-native architecture. By utilizing Azure's specialized storage and identity services alongside Python's backend capabilities, the project demonstrates how cloud technologies can streamline supply chain operations, enhance data security, and optimize performance.

- **Project Objectives**

The primary objective of this project is to build a Cloud-Based Inventory Management System that:

- Provides real-time tracking of inventory levels, item details, and stock movements.
- authenticates users and manages access using Azure Identity and cryptographic [PASSWORD] (bcrypt).
- ents automated barcode generation for quick item logging and retrieval.
- Ensures high availability and data redundancy using Microsoft Azure.

- **Scope of the Project**

The scope of this project focuses on designing and implementing the backend architecture for a cloud-native inventory platform. It encompasses configuring the Microsoft Azure environment and writing Python application logic to manage data flow across Azure Cosmos DB, Blob Storage, and Queue Storage. Additionally, the s mplements automated barcode generation for physical tracking and secure credential [PASSWORD] using Azure Identity and the bcrypt library. Serving as a practical Proof-of-Concept f Z-900 internship, the project demonstrates the successful orchestration of cloud connectivity, backend security, and database management.

2. Azure Cloud Architecture & Fundamentals

2.1 Cloud Computing Paradigm & AZ-900 Concepts The foundation of the Cloud-Based Inventory Management System (CBIMS) relies on Microsoft Azure's public cloud infrastructure. Concepts mastered during the Ethnus-Microsoft Azure Fundamentals (AZ-900) internship—such as High Availability (HA), scalability, fault tolerance, and the Shared Responsibility Model—serve as the guiding principles for this architecture. By utilizing Platform-as-a-Service (PaaS) solutions, the administrative burden of maintaining physical servers is eliminated, allowing for a focus solely on application logic and data management.

2.2 Azure Cosmos DB Azure Cosmos DB is utilized as the primary, globally distributed NoSQL database for the inventory system. The system integrates with this service using the azure-cosmos Python library. Cosmos DB was selected for its schema-agnostic data modeling, which easily accommodates varying attributes for different stock-keeping units (SKUs) without requiring complex migrations. It guarantees single-digit millisecond read and write latencies, ensuring real-time inventory tracking.

2.3 Azure Blob Storage To manage the unstructured data generated by the inventory system, Azure Blob Storage is implemented via the azure-storage-blob SDK. In a modern inventory system, items are often tracked with visual assets. Blob Storage provides a highly durable, cost-effective repository for product images, dynamically generated barcode graphics, and exported CSV/PDF reports.

2.4 Azure Queue Storage System performance and responsiveness are optimized through asynchronous message processing using Azure Queue Storage, integrated via the azure-storage-queue package. Instead of forcing the application to synchronously process heavy background tasks—such as sending low-stock email alerts or syncing with external supplier APIs—these tasks are pushed as messages to a queue. Background worker processes then consume these messages at their own pace, ensuring the main user interface remains highly responsive.

2.5 Security and Identity Management Security is enforced at both the cloud resource and local application levels. The azure-identity library is employed to handle secure, passwordless authentication to Azure services using DefaultAzureCredential, which suppo anaged Identities and Service Principals. This ensures that connection strings and secret [PASSWORD] are not hardcoded into the application.

3. System Implementation & Workflow

3.1 Development Environment & Dependencies

The application backend is developed in Python. The project isolates its dependencies within a virtual environment, ensuring consistency across development and production states. Alongside the Azure SDKs, the environment utilizes two critical third- ibraries: barcode for automated barcode generation and bcrypt for secure credential [PASSWORD]

3.2 Core Application Modules

The implementation of the system is divided into four primary modules:

- **Authentication & Access Control Module:** This module handles user login and registration. Passwords are never stored in plain text; instead, the bcrypt library is used to generate cryptographic hashes w re salting. This protects the system against unauthorized access and credential [PASSWORD] attacks.
- **Barcode Generation Module:** Physical inventory tracking requires a bridge to the digital system. Upon the registration of a new product, the barcode library dynamically generates a unique Code128 or EAN-13 barcode graphic.
- **Media & Storage Module:** Once a barcode is generated, or a user uploads a product photo, this module uploads the file directly to an Azure Blob Storage container. The resulting public or SAS (Shared Access Signature) URI is then returned to the application.
- **Inventory Data Module:** This acts as the central data controller. It constructs a JSON document containing the product name, SKU, quantity, threshold limits, and the Blob URIs for its images. This document is then committed to Azure Cosmos DB.

3.3 Database Design and Data Models

Unlike relational databases, the Cosmos DB implementation utilizes a document-based structure. A typical item document contains:

- id: A unique identifier (UUID).
- sku: The Stock Keeping Unit string.
- product_name: The descriptive name of the item.
- quantity: Current stock level (Integer).
- reorder_threshold: The minimum quantity allowed before an alert is triggered.
- barcode_url: The Azure Blob Storage URI linking to the generated barcode image.

3.4 Execution Workflow

The end-to-end operational workflow of the CBIMS operates as follows:

1. **Authentication:** A warehouse manager attempts to log in. The system fetches the stored hash from the database and verifies it using bcrypt.
2. **Item Registration:** The manager inputs details for a new inventory batch.
3. **Asset Generation:** The system uses the barcode library to create a visual barcode, uploads it via azure-storage-blob, and retrieves the URL.
4. **Data Persistence:** The item data and asset URLs are written to Azure Cosmos DB.
5. **Transaction Monitoring:** As items are checked out (scanned out), the quantity in Cosmos DB is decremented.
6. **Asynchronous Alerting:** If the decrement pushes the quantity below the reorder_threshold, the system pushes a JSON payload to azure-storage-queue. A separate background function reads this queue and triggers a reorder notification to the purchasing department.

4. CONCLUSION

The Cloud-Based Inventory Management System (CBIMS) project successfully achieves its objective of modernizing traditional stock tracking through a robust, cloud-native architecture. By leveraging Microsoft Azure's Platform-as-a-Service (PaaS) solutions, the system overcomes the limitations of legacy monolithic structures, ensuring high availability, seamless scalability, and fault tolerance.

Technically, the project demonstrates the powerful interoperability of Azure services. Azure Cosmos DB provides a schema-agnostic, low-latency foundation for managing diverse inventory catalogs. Azure Blob Storage cost-effectively houses unstructured assets, including product images and dynamically generated barcodes. Meanwhile, Azure Queue Storage successfully decouples resource-heavy background tasks—such as low-stock alerts—to maintain optimal application performance. System security is strictly enforced b ing Azure Identity for secure cloud access with the bcrypt library for local credential [PASSWORD]

Developed as the culmination of the Ethnus-Microsoft Azure Fundamentals (AZ-900) internship, this project provided invaluable hands-on experience in serverless backend orchestration. Future enhancements could include integrating interactive web interfaces, RESTful APIs for ERP synchronization, and Azure Machine Learning for predictive restocking. Ultimately, this project proves that adopting a cloud-first approach yields a highly scalable and secure solution for modern supply chain management.

REFERENCES

[1] E. Team, "AZ-900 Microsoft Azure Fundamentals Internship Guidelines and Training Modules", *Ethnus Educational Documentation*, vol. 1, no. 1, pp. 1-30, May 2026.

[2] M. Corporation, "Azure SDK for Python: Integrating Cosmos DB, Blob Storage, and Queue Storage", *Microsoft Developer Documentation*, vol. 12, no. 5, pp. 40-85, June 2026.

[3] M. Person_007 , "Microsoft Azure Core Cloud Services, Identity, and Architecture Fundamentals", *Microsoft Azure Publications*, vol. 8, no. 2, pp. 15-40, March 2026.

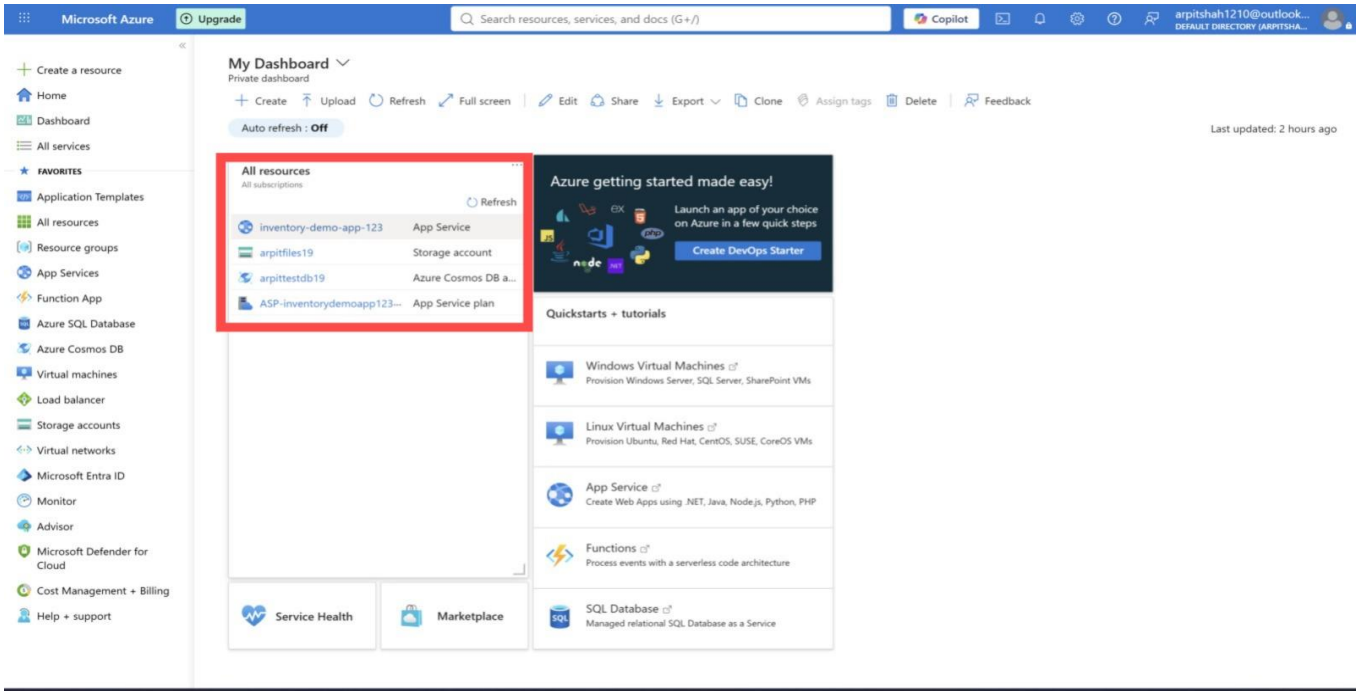
[4] P. S. Foundation, "Cryptographic Hashing and Barcode Generation in Python: Bcrypt and Barcode Libraries", *Python Package Index (PyPI) Documentation*, vol. 4, no. 8, pp. 100-115, Jan. 2026.

[5] R. Davis, K. Smith and J. Doe, "Serverless Architecture and Queue-Based Load Leveling for Enterprise Inventory Systems", IEEE Cloud Computing Magazine, vol. 11, no. 1, pp. 22-35, Jan. 2026.

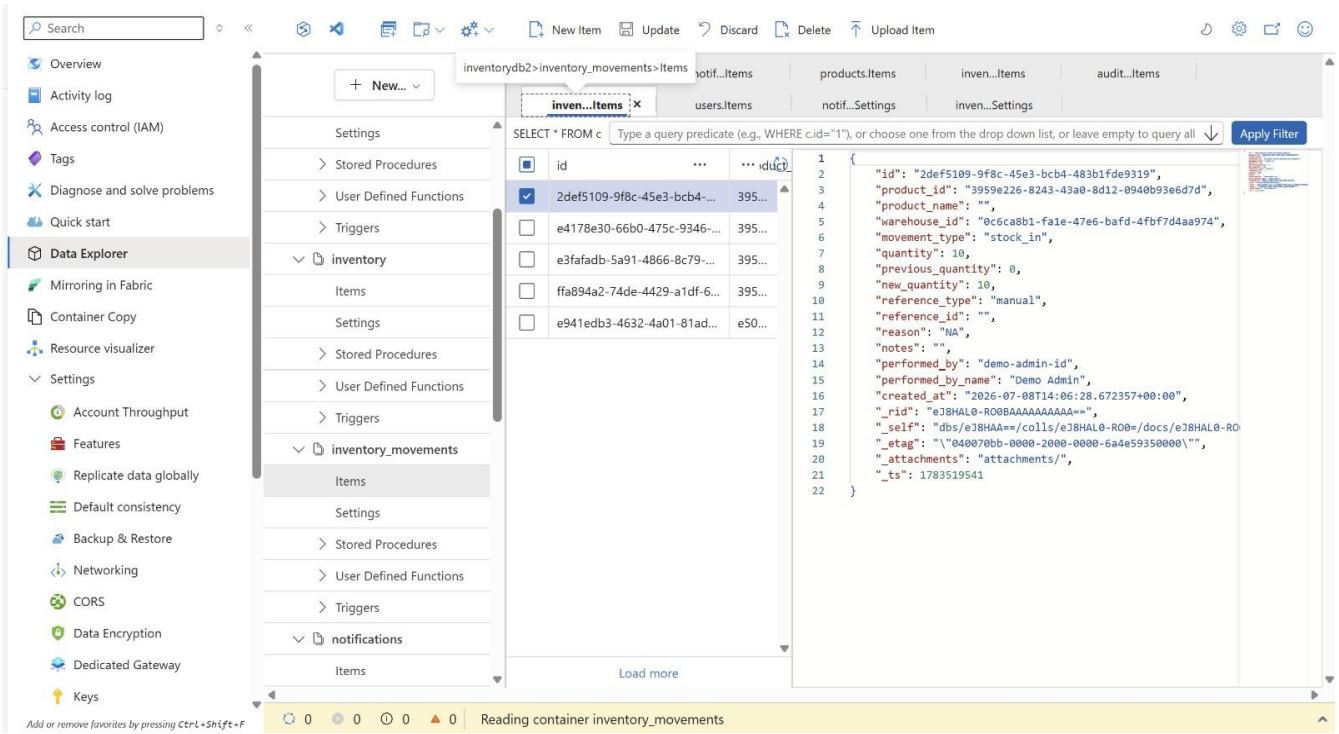
[6] S. Gupta and R. Patel, "Performance Analysis of Globally Distributed NoSQL Databases: A Case Study on Azure Cosmos DB", IEEE Transactions on Cloud Computing, vol. 14, no. 2, pp. 45-58, Feb. 2026.

APPENDIX I

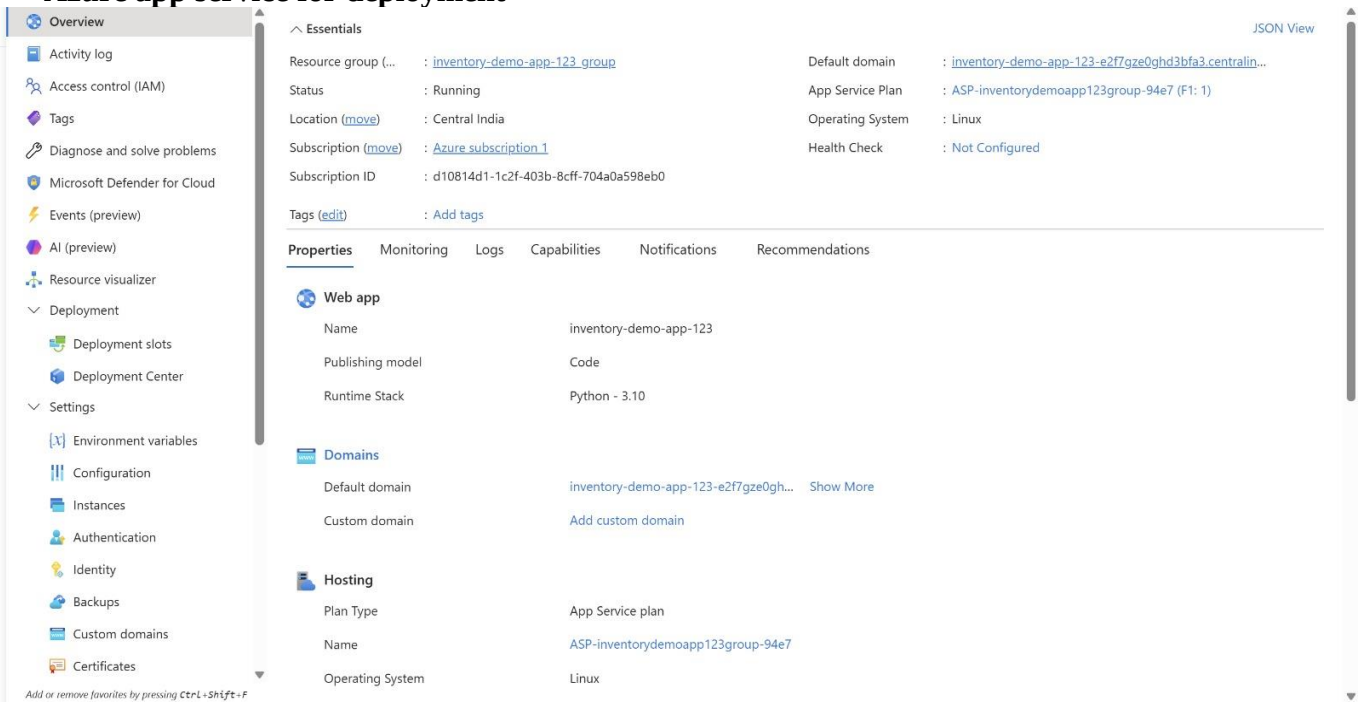
- **Azure Services used**



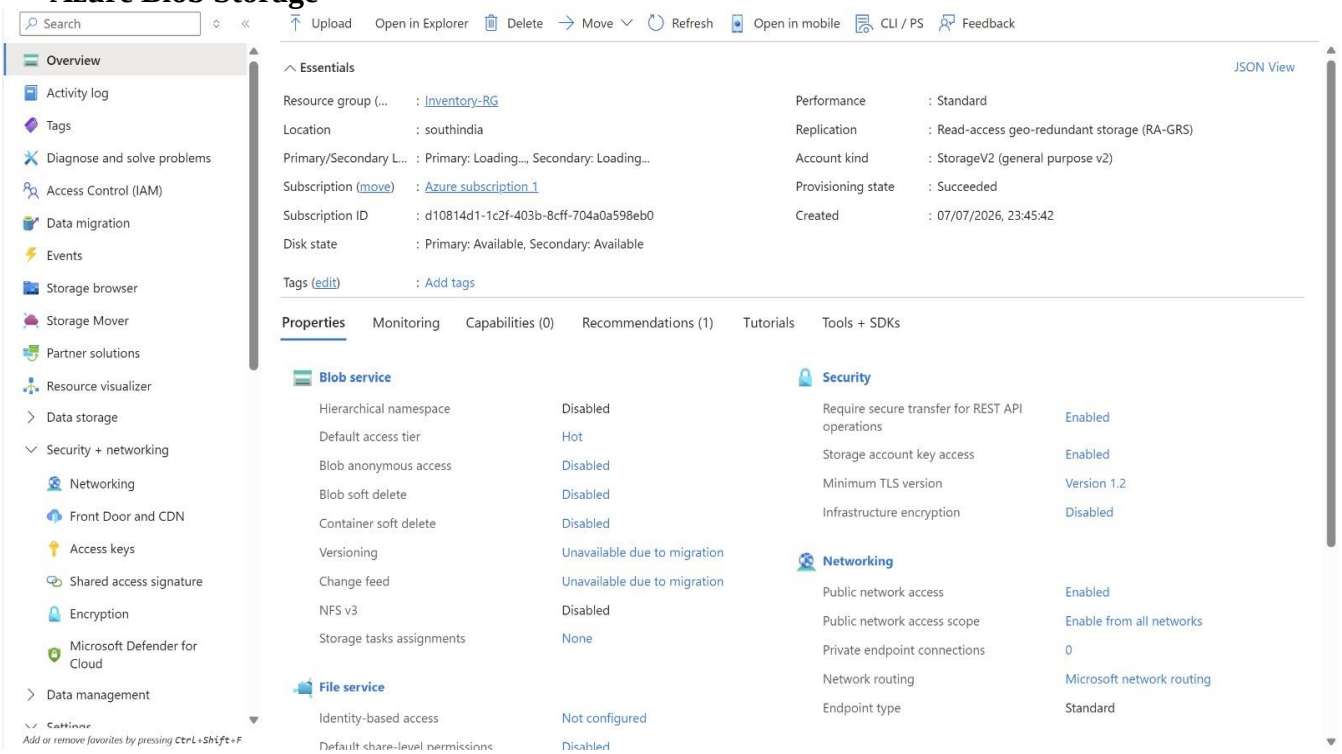
- **Azure Cosmos DB for backend**



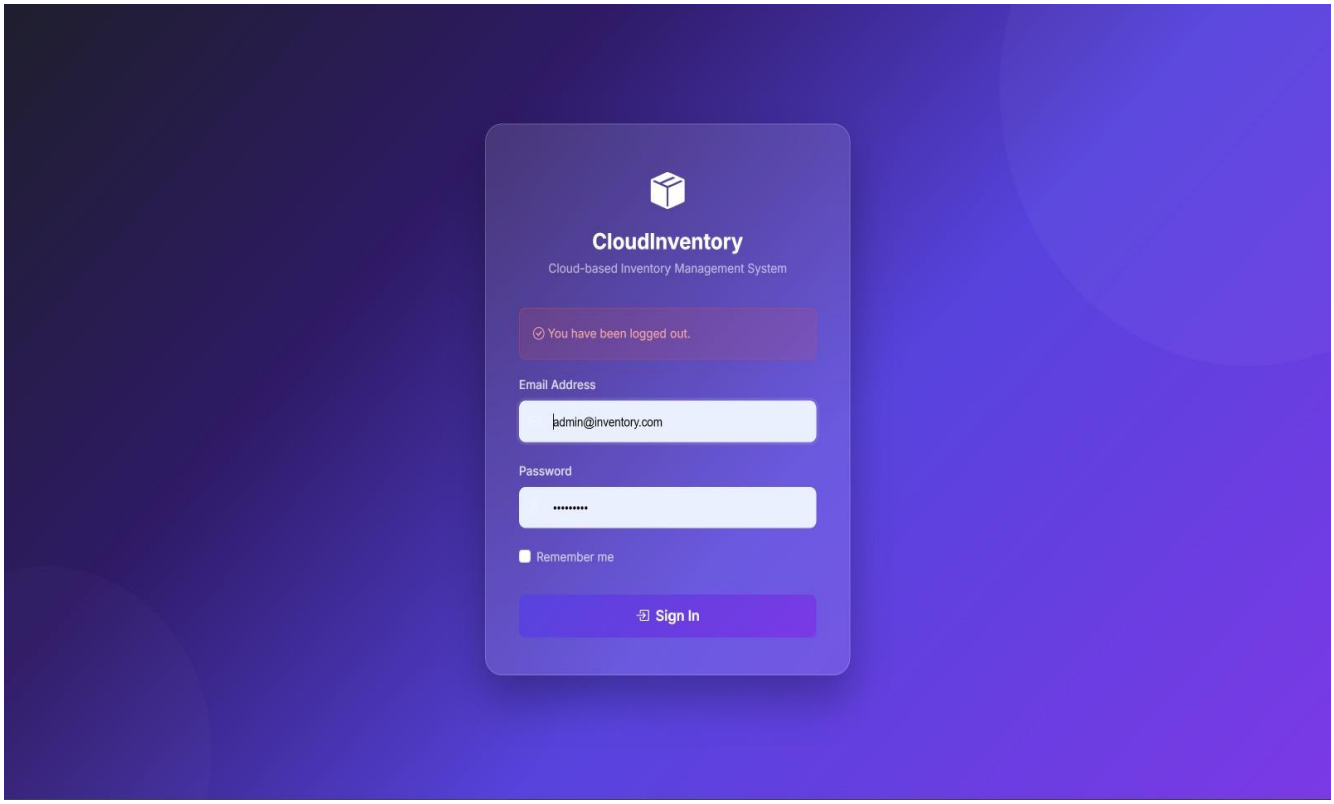
- **Azure app service for deployment**



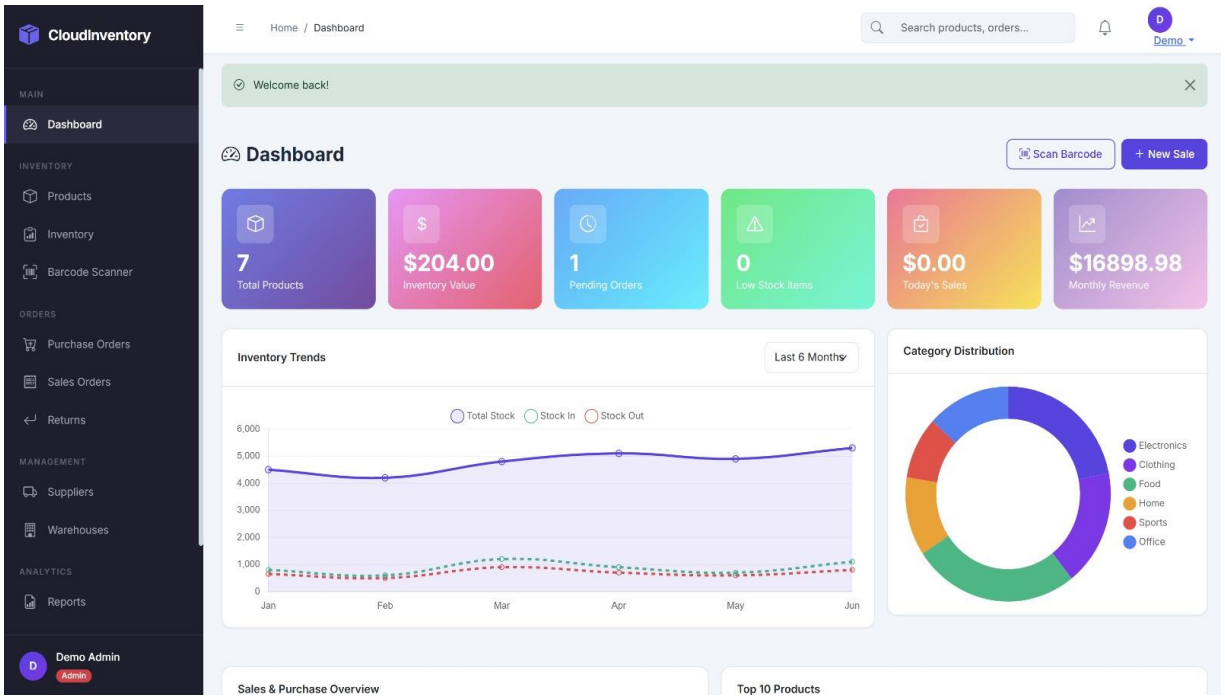
- **Azure Blob Storage**



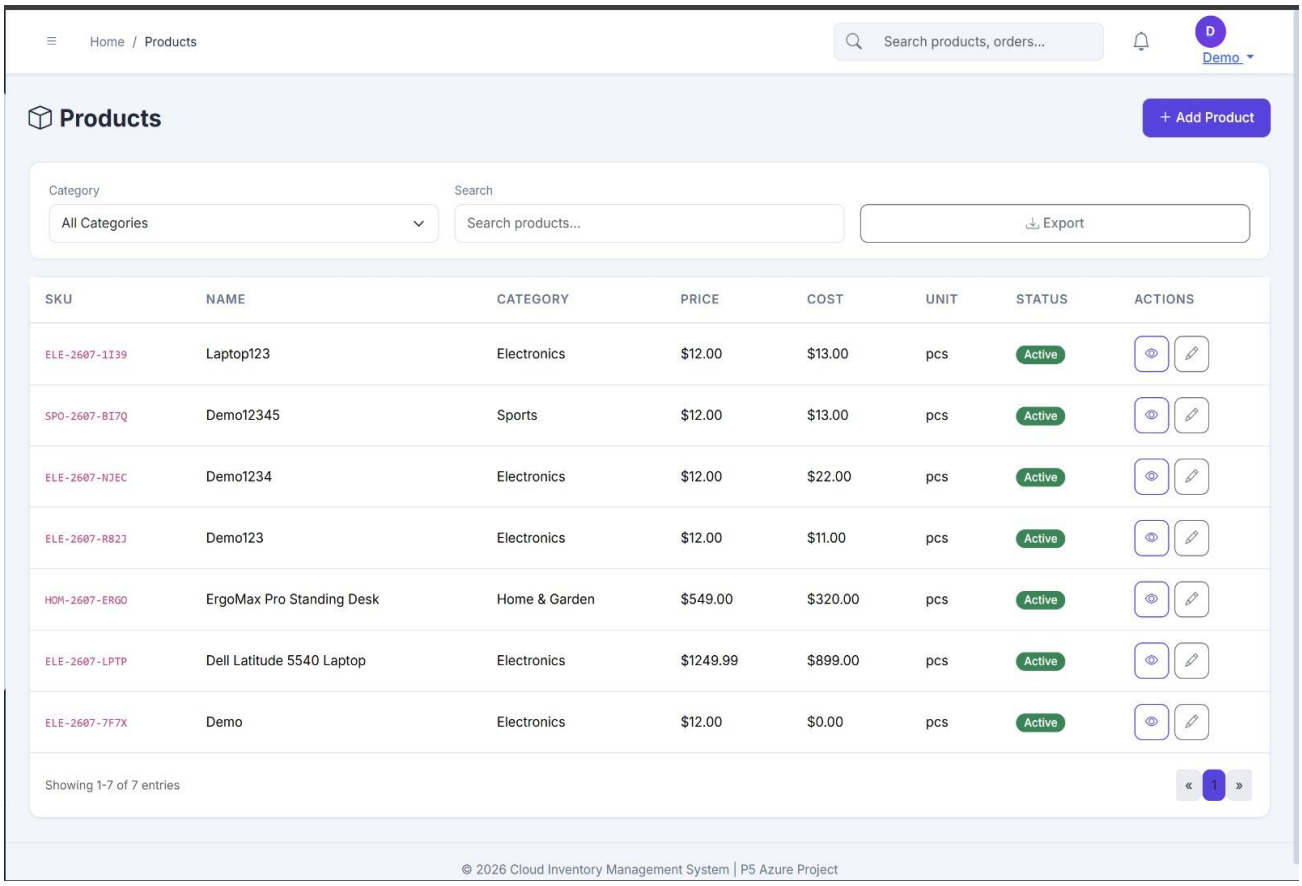
- **Login page**



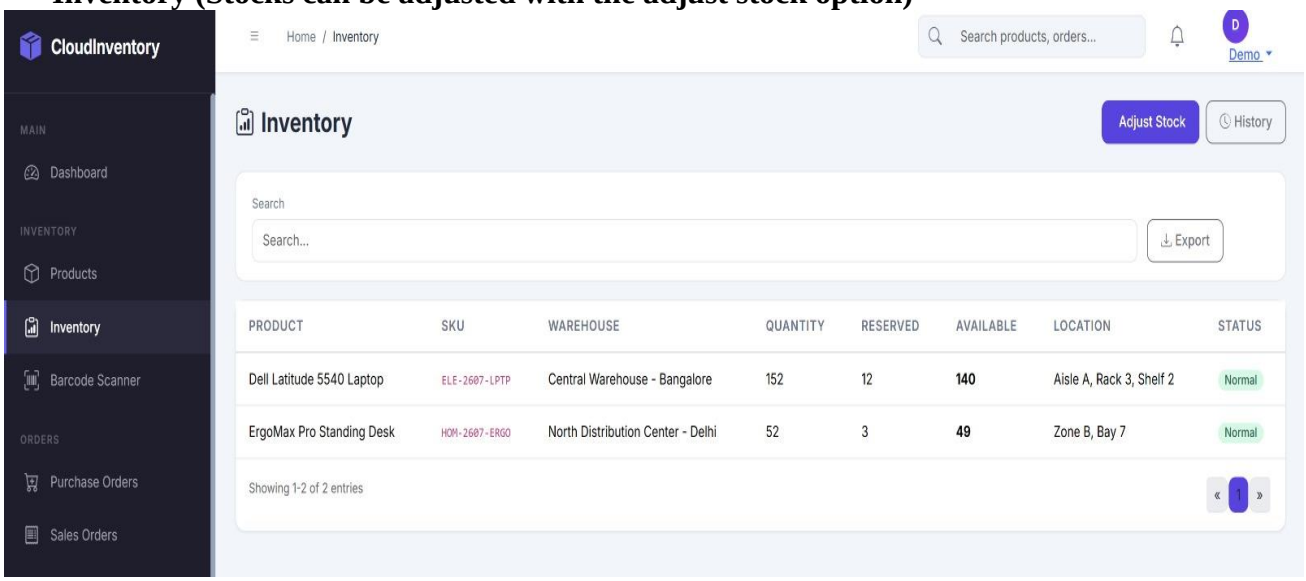
- **Dashboard**



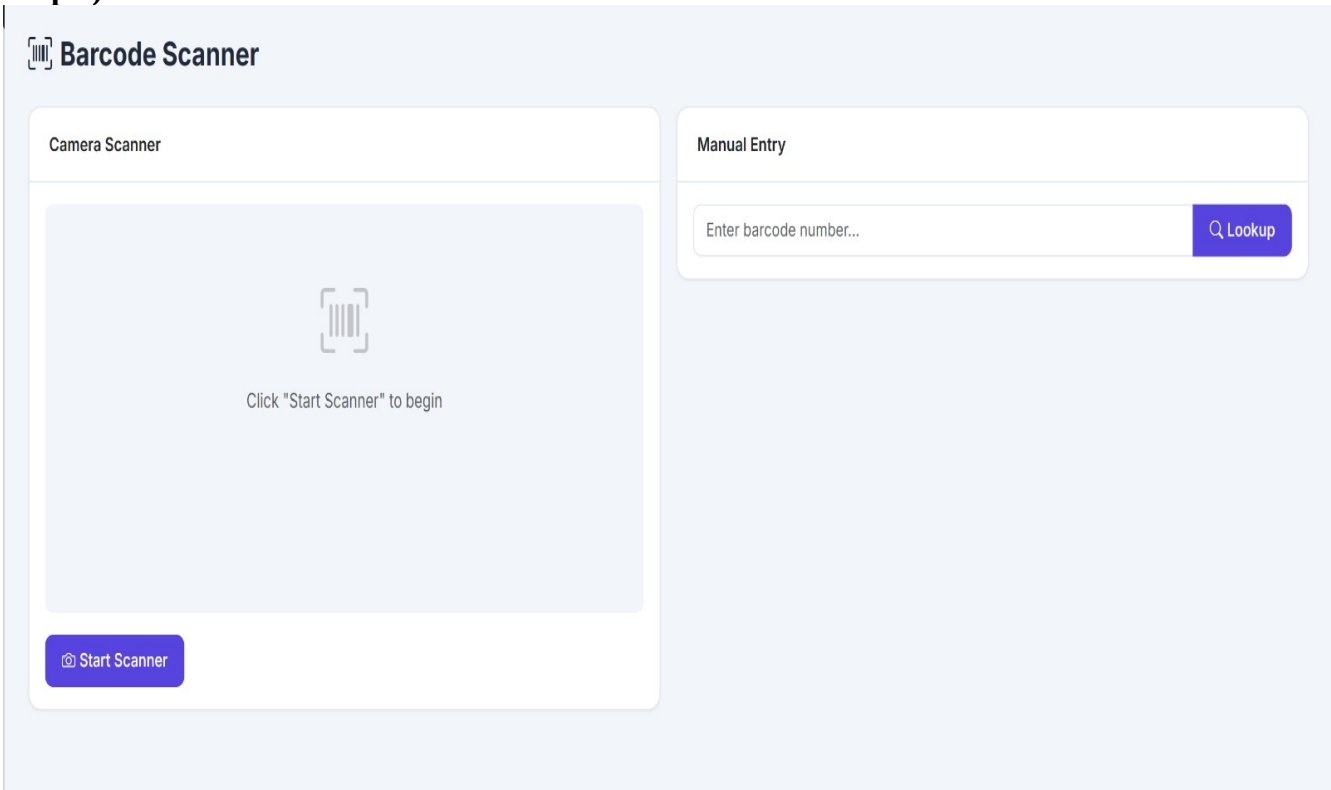
- **Products (Products can be added as well with the add product option)**



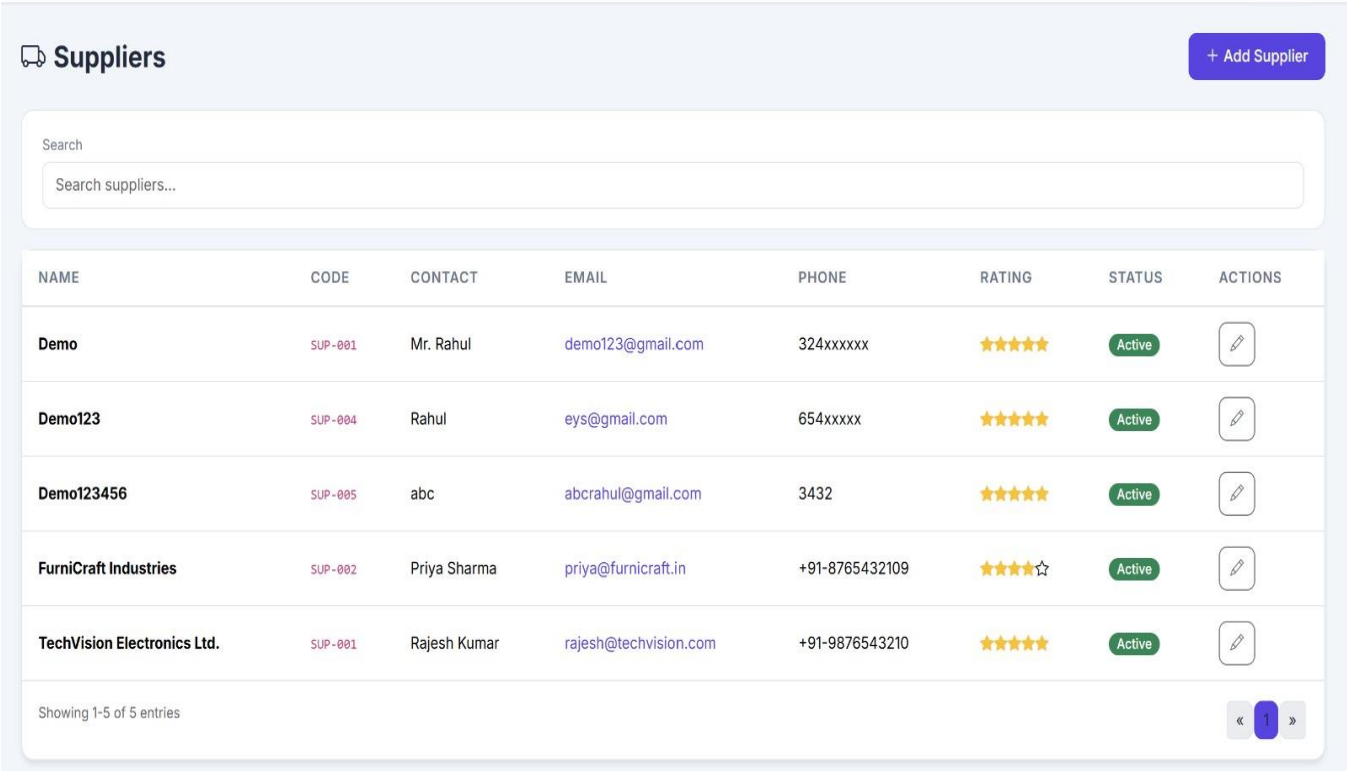
- **Inventory (Stocks can be adjusted with the adjust stock option)**



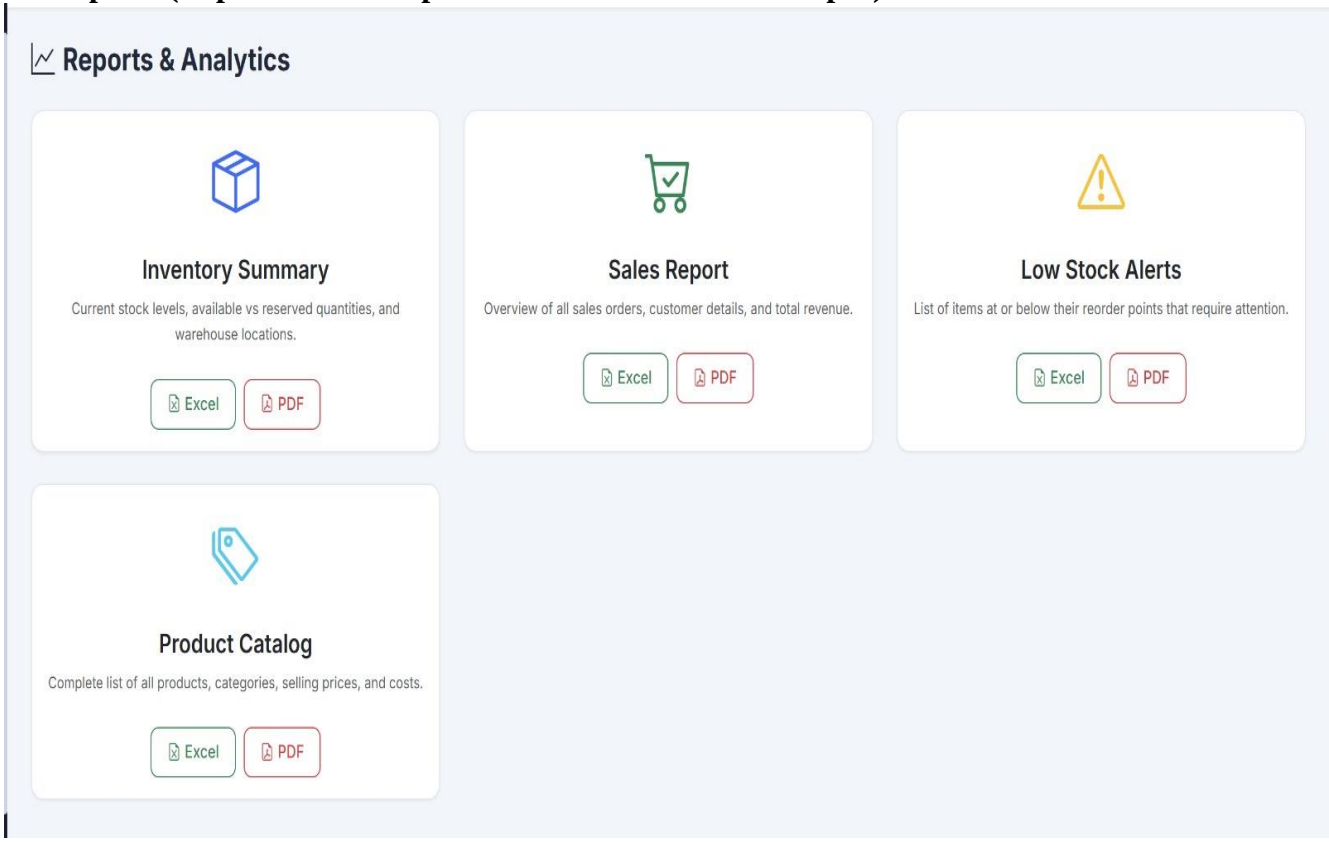
- **Barcode Scanner (Products can be searched here either manually or through barcode pic)**



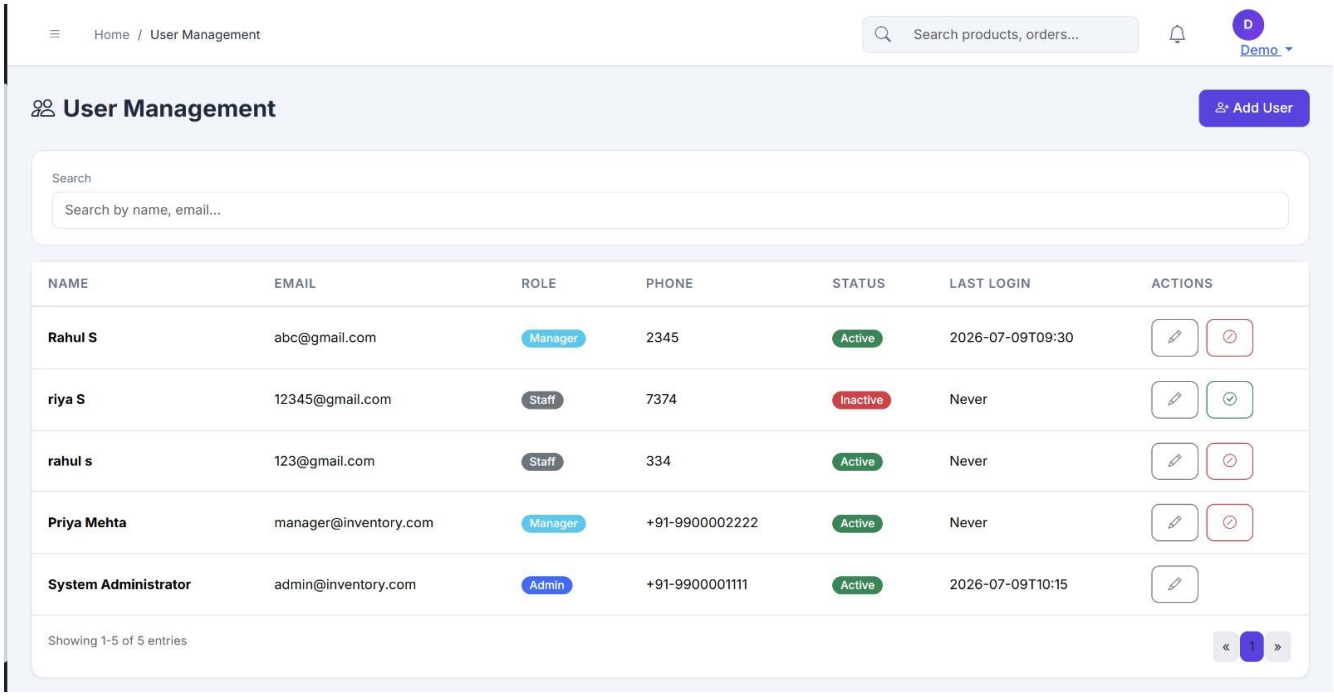
- **Suppliers**



- **Reports (Reports can be exported here either as excel or pdf)**



- **User Management (users can be added here and assigned roles such as admin, staff etc)**



- **Audit logs (who signed in/out, at what time etc is displayed here)**

